Книг ата за sdev

Пълно ръководство за програмния език sdev

Първо издание · 2026

Съдържание

Съдържание **3**

Книгата за sdev **16**

Съдържание 16

1. Въведение 20

1.1 Защо sdev? 21

1.2 Кратка история 21

1.3 За кого е тази книга 22

1.4 Как да използвате книгата 22

1.5 Конвенции 22

1.6 Първа глътка 22

2. Първи стъпки 23

2.1 Инсталация 23

2.2 Вашата първа програма 23

2.3 REPL 24

2.4 Структура на проекта 24

2.5 Стартиране от IDE 24

2.6 Анатомия на програма 24

2.7 Чести грешки за начинаещи 24

3. Променливи и стойности 25

3.1 Forge 25

3.2 Шест примитивни типа 25
3.3 Инспекция на тип 25
3.4 Преобразуване 25
3.5 Константи 26
3.6 Правила за именуване 26
3.7 Обхват 26
3.8 Истинност 26
3.9 Числа 26
3.10 Низове 26
4. Оператори и изрази 27
4.1 Аритметика 27
4.2 Сравнение 27
4.3 Логически 27
4.4 Низови 27
4.5 Принадлежност 27
4.6 Pipeline 28
4.7 Приоритет 28
5. Контролен поток 28
5.1 If/Then/Else 28
5.2 When 28
5.3 While и until 29
5.4 For ... in 29
5.5 Escape и skip 29
5.6 Етикетирани цикли 29

5.7 Match 29
6. Функции 29
6.1 Дефиниция 30
6.2 Аргументи по подразбиране 30
6.3 Variadic 30
6.4 Връщане 30
6.5 Ламбди 30
6.6 Затваряния 30
6.7 Higher-order 30
6.8 Рекурсия 31
6.9 Генератори 31
6.10 Декоратори 31
7. Колекции 31
7.1 Списъци 31
7.2 Речници 31
7.3 Срязване 32
7.4 Map/Filter/Reduce 32
7.5 Множества 32
7.6 Кортежи 32
7.7 Итерация 32
7.8 Производителност 32
8. Низове задълбочено 32
8.1 Литерали 33

8.2 Индексиране 33
8.3 Търсене 33
8.4 Манипулация 33
8.5 Кодиране 33
8.6 Форматиране 33
8.7 Регулярни изрази 33
8.8 Unicode 34
9. Обектно-ориентирано програмиране 34
9.1 Дефиниция на клас 34
9.2 Инстанции 34
9.3 Наследяване 34
9.4 Class методи 35
9.5 Mixins 35
9.6 Претоварване на оператори 35
9.7 Свойства 35
9.8 Абстрактни класове 35
10. Обработка на грешки 35
10.1 Raise 35
10.2 Attempt/Rescue/Finally 36
10.3 Типове грешки 36
10.4 Множество rescue 36
10.5 Re-raise 36
10.6 Stack traces 36
10.7 Asserts 36

10.8 Идиоми 37
11. Конкурентност 37
11.1 Async функции 37
11.2 Dispatch и await 37
11.3 Паралел 37
11.4 Канали 37
11.5 Отказване 38
11.6 Време 38
11.7 Ядро 38
12. Модули и библиотеки 38
12.1 Локални модули 38
12.2 Gist пакети 38
12.3 Версии 38
12.4 Манифести 39
12.5 Авторство 39
12.6 Обхват 39
12.7 Динамично зареждане 39
13. UI инструментариум 39
13.1 Прозорци 39
13.2 Заглавия и текст 39
13.3 Бутони 40
13.4 Полета 40
13.5 Многоредови полета 40

13.6 Чекбоксове и радиа 40
13.7 Слайдъри и select 40
13.8 Таблици 40
13.9 Tabs 41
13.10 Менюта 41
13.11 Подредба 41
13.12 Реактивни стойности 41
13.13 Събития 41
13.14 Персонализация 41
13.15 Пълен пример 41
14. Графика и canvas 42
14.1 Canvas 42
14.2 Примитиви 42
14.3 Цветове 42
14.4 Анимация 42
14.5 Изображения 42
14.6 Трансформации 43
14.7 Вход 43
14.8 Подскачаща топка 43
15. Карти и ГИС 43
15.1 Карта 43
15.2 Маркери 43
15.3 Tile слоеве 44
15.4 Полилинии 44

15.5 Полигони 44
15.6 GeoJSON 44
15.7 Heatmaps 44
15.8 Събития 44
15.9 Маршрутиране 44
16. Стандартна библиотека 45
16.1 Таблица 45
16.2 I/O 46
16.3 Математика 46
16.4 Колекции 46
16.5 Низове 47
16.6 Време 47
16.7 Мрежа 47
16.8 Система 47
16.9 Кодиране 47
16.10 Криптография 47
17. Инструменти 47
17.1 Уеб IDE 48
17.2 VS Code разширение 48
17.3 CLI 48
17.4 Bytecode компилатор 48
17.5 sdev асистент 48
17.6 Линтер 48
17.7 Форматер 48

17.8 Дебъгер 49
17.9 Тестове 49
18. Напреднали теми 49
18.1 JavaScript Interop 49
18.2 Матрична математика 49
18.3 Буфери и указатели 49
18.4 Виртуално ядро 49
18.5 Сметоизхвърляне 50
18.6 Bytecode VM 50
18.7 Многоезичен код 50
18.8 Вграждане 50
18.9 FFI 50
19. Производителност 50
19.1 Изберете рънтайм 51
19.2 Профилирайте 51
19.3 Избягвайте квадратични шаблони 51
19.4 Кеширане 51
19.5 Преизползвайте буфери 51
19.6 Компилирайте 51
20. Добри практики 51
20.1 Именувайте добре 52
20.2 Предпочитайте глаголи 52
20.3 Малки функции 52

20.4 Прегърнете неизменността 52
20.5 Документирайте намерението 52
20.6 Тествайте рано 52
20.7 Проваляйте се шумно 52
20.8 Композиция над наследяване 53
21. Готварска книга с примери 53
21.1 FizzBuzz 53
21.2 Брояч на думи 53
21.3 Хистограма 53
21.4 Todo приложение 54
21.5 HTTP заявка 54
21.6 Подскачаща топка 54
21.7 Палиндром 54
21.8 Фибоначи 55
21.9 Калкулатор UI 55
21.10 Карта на столици 55
22. Изграждане на реално приложение 55
22.1 Подредба 56
22.2 Storage модул 56
22.3 Render модул 56
22.4 UI 56
22.5 Свързване 56
22.6 Облак 56
22.7 Публикуване 57

23. Често задавани въпроси 57
23.1 Бърз ли е sdev? 57
23.2 Може ли в продукция? 57
23.3 Защо не Python? 57
23.4 Защо два рънтайма? 57
23.5 Бъгове? 58
23.6 Принос? 58
23.7 Запазване между сесии? 58
23.8 Защо "forge"? 58
24. Речник 58
24.1 А–Й 58
24.2 К–Р 59
24.3 С–Я 59
25. Бележки за дизайна 59
25.1 Глаголи над символи 59
25.2 Защо end 59
25.3 Два рънтайма 60
25.4 UI вграден 60
25.5 Многоезичен 60
25.6 Вдъхновения 60
26. Приложения 60
26.1 A. Ключови думи 60
26.2 B. Оператори 61

26.3 C. Вградени функции 61
26.4 D. Изходни езици 61
26.5 E. Bytecode опкодове 61
26.6 F. Syscalls 61
26.7 G. Кодове на грешки 62
26.8 H. CLI флагове 62
26.9 I. Индекс на примерите 62

ГЛАВА 01

Книгата за sdev

Книгата за sdev

Изчерпателен наръчник за програмния език sdev.

Съдържание

1. Въведение

- 1.1 Защо sdev? - 1.2 Кратка история - 1.3 За кого е тази книга - 1.4 Как да използвате книгата - 1.5 Конвенции - 1.6 Първа глътка

2. Първи стъпки

- 2.1 Инсталация - 2.2 Вашата първа програма - 2.3 REPL - 2.4 Структура на проекта - 2.5 Стартиране от IDE - 2.6 Анатомия на програма - 2.7 Чести грешки за начинаещи

3. Променливи и стойности

- 3.1 Forge - 3.2 Шест примитивни типа - 3.3 Инспекция на тип - 3.4 Преобразуване - 3.5 Константи - 3.6 Правила за именуване - 3.7 Обхват - 3.8 Истинност - 3.9 Числа - 3.10 Низове

4. Оператори и изрази

- 4.1 Аритметика - 4.2 Сравнение - 4.3 Логически - 4.4 Низови - 4.5 Принадлежност - 4.6 Pipeline - 4.7 Приоритет

5. Контролен поток

- 5.1 If/Then/Else - 5.2 When - 5.3 While и until - 5.4 For ... in - 5.5 Escape и skip - 5.6 Етикетирани цикли - 5.7 Match

6. Функции

- 6.1 Дефиниция - 6.2 Аргументи по подразбиране - 6.3 Variadic - 6.4 Връщане - 6.5 Ламбди - 6.6 Затваряния - 6.7 Higher-order - 6.8 Рекурсия - 6.9 Генератори - 6.10 Декоратори

7. Колекции

- 7.1 Списъци - 7.2 Речници - 7.3 Срязване - 7.4 Map/Filter/Reduce - 7.5 Множества - 7.6 Кортежи - 7.7 Итерация - 7.8 Производителност

8. Низове задълбочено

- 8.1 Литерали - 8.2 Индексиране - 8.3 Търсене - 8.4 Манипулация - 8.5 Кодиране - 8.6 Форматиране - 8.7 Регулярни изрази - 8.8 Unicode

9. Обектно-ориентирано програмиране

- 9.1 Дефиниция на клас - 9.2 Инстанции - 9.3 Наследяване - 9.4 Class методи - 9.5 Mixins - 9.6 Претоварване на оператори - 9.7 Свойства - 9.8 Абстрактни класове

10. Обработка на грешки

- 10.1 Raise - 10.2 Attempt/Rescue/Finally - 10.3 Типове грешки - 10.4 Множество rescue - 10.5 Re-raise - 10.6 Stack traces - 10.7 Asserts - 10.8 Идиоми

11. Конкурентност

- 11.1 Async функции - 11.2 Dispatch и await - 11.3 Паралел - 11.4 Канали - 11.5 Отказване - 11.6 Време - 11.7 Ядро

12. Модули и библиотеки

- 12.1 Локални модули - 12.2 Gist пакети - 12.3 Версии - 12.4 Манифести - 12.5 Авторство - 12.6 Обхват - 12.7 Динамично зареждане

13. UI инструментариум

- 13.1 Прозорци - 13.2 Заглавия и текст - 13.3 Бутони - 13.4 Полета - 13.5 Многоредови полета - 13.6 Чекбоксове и радиа - 13.7 Слайдъри и select - 13.8 Таблици - 13.9 Tabs - 13.10 Менюта - 13.11 Подредба - 13.12 Реактивни стойности - 13.13 Събития - 13.14 Персонализация - 13.15 Пълен пример

14. Графика и canvas

- 14.1 Canvas - 14.2 Примитиви - 14.3 Цветове - 14.4 Анимация - 14.5 Изображения - 14.6 Трансформации - 14.7 Вход - 14.8 Подскачаща топка

15. Карти и ГИС

- 15.1 Карта - 15.2 Маркери - 15.3 Tile слоеве - 15.4 Полилинии - 15.5 Полигони - 15.6 GeoJSON - 15.7 Heatmaps - 15.8 Събития - 15.9 Маршрутиране

16. Стандартна библиотека

- 16.1 Таблица - 16.2 I/O - 16.3 Математика - 16.4 Колекции - 16.5 Низове - 16.6 Време - 16.7 Мрежа - 16.8 Система - 16.9 Кодиране - 16.10 Криптография

17. Инструменти

- 17.1 Уеб IDE - 17.2 VS Code разширение - 17.3 CLI - 17.4 Bytecode компилатор - 17.5 sdev асистент - 17.6 Линтер - 17.7 Форматер - 17.8 Дебъгер - 17.9 Тестове

18. Напреднали теми

- 18.1 JavaScript Interop - 18.2 Матрична математика - 18.3 Буфери и указатели - 18.4 Виртуално ядро - 18.5 Сметоизхвърляне - 18.6 Bytecode VM - 18.7 Многоезичен код - 18.8 Вграждане - 18.9 FFI

19. Производителност

- 19.1 Изберете рънтайм - 19.2 Профилирайте - 19.3 Избягвайте квадратични шаблони - 19.4 Кеширане - 19.5 Преизползвайте буфери - 19.6 Компилирайте

20. Добри практики

- 20.1 Именувайте добре - 20.2 Предпочитайте глаголи - 20.3 Малки функции - 20.4 Прегърнете неизменността - 20.5 Документирайте намерението - 20.6 Тествайте рано - 20.7 Проваляйте се шумно - 20.8 Композиция над наследяване

21. Готварска книга с примери

- 21.1 FizzBuzz - 21.2 Брояч на думи - 21.3 Хистограма - 21.4 Todo приложение - 21.5 HTTP заявка - 21.6 Подскачаща топка - 21.7 Палиндром - 21.8 Фибоначи - 21.9 Калкулатор UI - 21.10 Карта на столици

22. Изграждане на реално приложение

- 22.1 Подредба - 22.2 Storage модул - 22.3 Render модул - 22.4 UI - 22.5 Свързване - 22.6 Облак - 22.7

Публикуване

23. Често задавани въпроси

- 23.1 Бърз ли е sdev? - 23.2 Може ли в продукция? - 23.3 Защо не Python? - 23.4 Защо два рънтайма? - 23.5 Бъгове? - 23.6 Принос? - 23.7 Запазване между сесии? - 23.8 Защо "forge"?

24. Речник

- 24.1 А–Й - 24.2 К–Р - 24.3 С–Я

25. Бележки за дизайна

- 25.1 Глаголи над символи - 25.2 Защо □CODE0□ - 25.3 Два рънтайма - 25.4 UI вграден - 25.5 Многоезичен - 25.6 Вдъхновения

26. Приложения

- 26.1 A. Ключови думи - 26.2 B. Оператори - 26.3 C. Вградени функции - 26.4 D. Изходни езици - 26.5 E. Bytecode опкодове - 26.6 F. Syscalls - 26.7 G. Кодове на грешки - 26.8 H. CLI флагове - 26.9 I. Индекс на примерите

1. Въведение

Добре дошли в **Книгата за sdev** — окончателният, изчерпателен наръчник за програмния език sdev. sdev е модерен, изразителен език с уникално четим синтаксис (`forge`, `be`, `speak`, `conjure`, `summon`), създаден да направи кода да изглежда като проза, без да

жертва мощта, скоростта или изразителността.

Тази книга е *дълга*. Това е нарочно. Програмните езици са богати светове, а sdev — с четимия си синтаксис, два различни рънтайма, UI инструментариум, ядро, байткод VM и поддръжка на 26 езика — има повече повърхност от повечето.

1.1 Защо sdev?

sdev е създаден, за да преодолее пропастта между *говоренето за код* и *писането на код*. Класическите езици използват алгебрични съкращения (`x = 1`, `def`, `var`). sdev ги заменя с глаголи и съществителни, които носят смисъл: `forge x be 1`. Резултатът е код, който се чете като рецепта или есе.

sdev също приема сериозно идеята, че **един език трябва да идва с всичко необходимо**: UI инструментариум, графичен canvas, картографски API, планировчик в стил ядро, байткод VM, AI инструменти и преводи на ключовите думи на 26 езика.

1.2 Кратка история

sdev започна като експеримент в *проза-подобно* програмиране. Първият интерпретатор беше 400-редов tree-walker. От там се разрасна в два паралелни рънтайма (JavaScript и Python), байткод

компилатор, виртуално ядро и IDE в браузъра.

1.3 За кого е тази книга

Тази книга е за всички: пълни начинаещи, професионални програмисти, преподаватели и дизайнери на езици. Всяка глава започва меко и постепенно усложнява материала.

1.4 Как да използвате книгата

Прочетете я последователно първия път. Всеки пример може да бъде стартиран в **онлайн IDE** на https://s-dev.lovable.app или с офлайн интерпретатори. Пишете примерите — не само ги четете.

1.5 Конвенции

Кодовите блокове използват синтаксиса `sdev`. Изходът е показан в `// =>` коментари. Бележки **СЪВЕТ**, **ВНИМАНИЕ** и **ЗАДЪЛБОЧЕНО** маркират важна информация.

1.6 Първа глътка

```
forge greet be conjure(name): "Здравей, " + name end
speak(greet("свят"))
// =&gt; Здравей, свят
```

2. Първи стъпки

В тази глава ще инсталирате sdev (или ще пропуснете инсталацията, използвайки уеб IDE), ще напишете първата си програма и ще разберете компонентите на sdev проект.

2.1 Инсталация

Има няколко начина за стартиране на sdev:

1. **Уеб IDE** — без инсталация, отворете https://s-dev.lovable.app/ide.
2. **JavaScript интерпретатор** — `node sdev-interpreter.js моят-файл.sdev`.
3. **Python интерпретатор** — `python sdev-interpreter.py моят-файл.sdev`.
4. **VS Code разширение** — `.vsix` от менюто Downloads.
5. **Windows инсталатор** — пакетен инсталатор с миниатюрен редактор.

2.2 Вашата първа програма

```
speak("Здравей, свят!")
```

Стартирайте го. Поздравления — вие сте sdev програмист.

2.3 REPL

Стартирайте REPL с `python sdev-interpreter.py` без аргументи. Въвеждайте изрази и те ще бъдат изчислявани веднага. Команди: `:quit`, `:reset`, `:load`, `:save`, `:type`.

2.4 Структура на проекта

Типичен sdev проект има `main.sdev`, папка `lib/` с модули, папка `assets/`, и `README.md`. Няма задължително оформление.

2.5 Стартиране от IDE

В уеб IDE натиснете `Ctrl+Enter` (или `Cmd+Enter`) за изпълнение. Output показва текста; App панелът показва UI; Canvas показва графика; Map показва Leaflet.

2.6 Анатомия на програма

Един sdev файл е поредица от изрази. Включва декларации, контролни структури, дефиниции на функции и класове, и импорти. Отстъпите са за хората; `end` е за парсера.

2.7 Чести грешки за начинаещи

- Забравяне на `end` след `if`, `while` или `conjure`.

- Използване на `=` вместо `be`. И двете работят; `be` е каноничното.
- Смесване на `yes`/`no` с `true`/`false`. Изберете един стил.

3. Променливи и стойности

В sdev създавате променливи с `forge` и ги обвързвате с `be`. Стойностите са динамично типизирани.

3.1 Forge

```
forge name be "Ада"
forge age be 36
```

Преобвързване без `forge` запазва променливата в първоначалния обхват.

3.2 Шест примитивни типа

`number`, `string`, `boolean` (`yes`/`no`), `void` (`nothing`), `collection`, `function`.

3.3 Инспекция на тип

`essence(value)` връща типа като низ.

3.4 Преобразуване

`morph(value, "type")` преобразува типове.

3.5 Константи

`eternal forge PI be 3.14159` — не може да бъде преобвързано.

3.6 Правила за именуване

Идентификаторите започват с буква или подчертавка. snake_case за променливи, PascalCase за класове.

3.7 Обхват

Лексикален обхват. Функциите образуват затваряния (closures).

3.8 Истинност

Falsy: `no`, `nothing`, `0`, `""`, празна колекция.

3.9 Числа

64-битови с плаваща запетая с бърз път за цели числа. `bigint(n)` за произволна точност.

3.10 Низове

Неизменни Unicode. Тройни кавички запазват редовете. Backtick поддържа `${интерполация}`.

4. Оператори и изрази

sdev поддържа стандартни оператори с английски псевдоними.

4.1 Аритметика

`+`, `-`, `*`, `/`, `%`, `**`, `//`.

4.2 Сравнение

`==`, `!=`, `<`, `>`, `<=`, `>=`. Псевдоними: `equals`, `differs`, `above`, `below`.

4.3 Логически

`and`, `or`, `not` (с късо съединение).

4.4 Низови

Конкатенация с `+`. Повторение с `*`. Интерполация с backtick.

4.5 Принадлежност

`x in coll`.

4.6 Pipeline

`" hi " |> trim |> upper |> speak`.

4.7 Приоритет

От най-стегнат до най-слаб: `**`, унарен, `* / // %`, `+ -`, сравнения, `and`, `or`, `|>`, присвояване.

5. Контролен поток

sdev предлага познати контролни структури с английски привкус.

5.1 If/Then/Else

```
if score above 90 then
  speak("Отлично")
else
  speak("Опитай пак")
end
```

5.2 When

```
when:
  is x below 0 then speak("отрицателно")
  is x equals 0 then speak("нула")
  else speak("положително")
```

```
end
```

5.3 While и until

```
while count below 10:
  count be count + 1
end
```

5.4 For ... in

```
for name in ["Ада", "Грейс"]: speak(name) end
```

5.5 Escape и skip

`escape` напуска цикъл; `skip` прескача итерация.

5.6 Етикетирани цикли

`label name:` + `escape name` за вложени цикли.

5.7 Match

Деструктуриране: `forge [a, *rest] be [1,2,3]`.

6. Функции

Функциите са обекти от първи клас.

6.1 Дефиниция

```
conjure greet(name be "непознат"):
  speak("Здравей, " + name)
end
```

6.2 Аргументи по подразбиране

Стойности с `be`. Извикване с ключови думи: `greet(name: "Ада")`.

6.3 Variadic

`*items` и `**opts`.

6.4 Връщане

`return value` за ранен изход.

6.5 Ламбди

`forge double be (n) => n * 2`.

6.6 Затваряния

Функциите улавят външни променливи.

6.7 Higher-order

`each`, `sift`, `fold`, `seek`.

6.8 Рекурсия

Поддържа неограничена рекурсия. `tailcall` модификатор за tail-call оптимизация.

6.9 Генератори

Функции с `yield` връщат итератори.

6.10 Декоратори

`@logged conjure work(): ... end`.

7. Колекции

Списъци и речници.

7.1 Списъци

```
forge nums be [1, 2, 3]
speak(nums[0])
gather(nums, 4)
```

7.2 Речници

```
forge user be {name: "Ада", age: 36}
speak(user.name)
```

7.3 Срязване

`portion(coll, start, end?)`.

7.4 Map/Filter/Reduce

`each`, `sift`, `fold`.

7.5 Множества

`unique(coll)`, `set(coll)`.

7.6 Кортежи

`(a, b)` — неизменни.

7.7 Итерация

`for ... in`, `enumerate`, `zip`.

7.8 Производителност

Списъци: O(1) добавяне. Речници: O(1) търсене. `pluck(coll, 0)` е O(n).

8. Низове задълбочено

Текстът е навсякъде — заслужава отделна глава.

8.1 Литерали

`''`, `""`, `"""..."""`, и backtick за интерполация.

8.2 Индексиране

Низовете са индексируеми. Неизменни.

8.3 Търсене

`contains`, `indexOf`, `regex`.

8.4 Манипулация

`upper`, `lower`, `trim`, `replace`, `shatter`, `weave`.

8.5 Кодиране

`base64encode`, `urlencode`, `hexencode`.

8.6 Форматиране

Backtick интерполация или `format`.

8.7 Регулярни изрази

`regex(pattern, s)` връща обект с `groups`.

8.8 Unicode

Низовете са последователности от Unicode codepoints. `locale("bg_BG")` за локализирани операции.

9. Обектно-ориентирано програмиране

Класове с `essence`, инстанции със `summon`, наследяване с `extends`.

9.1 Дефиниция на клас

```
essence Dog:
  share name
  conjure init(name): me.name be name end
  conjure bark(): speak(me.name + " лае!") end
end
```

9.2 Инстанции

```
forge rex be summon Dog(name: "Рекс")
rex.bark()
```

9.3 Наследяване

`essence Puppy extends Dog: ... end`. `ancestor.method()` извиква базовия.

9.4 Class методи

`static share` и `static conjure`.

9.5 Mixins

`essence Bird weave Flyable, Singable: end`.

9.6 Претоварване на оператори

`__add__`, `__eq__`, `__str__`.

9.7 Свойства

`@property` и `@setter`.

9.8 Абстрактни класове

`abstract conjure name()`.

10. Обработка на грешки

Програмите се провалят. sdev предлага точни инструменти.

10.1 Raise

`raise "Без гориво"` или `raise FuelError(...)`.

10.2 Attempt/Rescue/Finally

```
attempt:
  forge data be readFile("missing.txt")
rescue err:
  speak(err.message)
finally:
  speak("край")
end
```

10.3 Типове грешки

`TypeError`, `ValueError`, `IOError`, `KeyError`, `IndexError`, `NetworkError`, `TimeoutError`.

10.4 Множество rescue

Свързани с тип на грешката.

10.5 Re-raise

`raise` без аргумент в `rescue`.

10.6 Stack traces

Грешките имат `.trace` с рамки.

10.7 Asserts

`ensure(условие, "съобщение")`.

10.8 Идиоми

Валидирайте на границите. Не превръщайте rescue в тих поглъщач.

11. Конкурентност

Кооперативна конкурентност.

11.1 Async функции

`async conjure ...` + `await`.

11.2 Dispatch и await

`forge p be dispatch f()` + `await p`.

11.3 Паралел

`await all([...])`, `await any([...])`, `await race([...])`.

11.4 Канали

`channel()`, `send`, `receive`.

11.5 Отказване

`cancel(p)`, `rescue CancelledError`.

11.6 Време

`await delay(1000)`, `every(ms, fn)`.

11.7 Ядро

Всички задачи живеят във виртуално ядро.

12. Модули и библиотеки

Реалните програми се състоят от много файлове.

12.1 Локални модули

`summon "./utils.sdev" as u`.

12.2 Gist пакети

Декентрализиран регистър чрез GitHub Gists.

12.3 Версии

`summon "gist:abc@v1.2.0/util.sdev"`.

12.4 Манифести

Файл `manifest.sdev` декларира зависимости.

12.5 Авторство

Публични `conjure` и `essence` са експортирани. `share private` скрива.

12.6 Обхват

Всеки модул има собствен глобален обхват.

12.7 Динамично зареждане

`loadModule(url)`.

13. UI инструментариум

Декларативен UI: прозорци, форми, дашбордове.

13.1 Прозорци

```
window("Моето приложение", 480, 600)
  heading("Здравей", 1)
endwindow
```

13.2 Заглавия и текст

`heading(text, level)`, `label(text)`, `paragraph(text)`.

13.3 Бутони

`button("Натисни", () => ..., "primary")`. Варианти: primary, secondary, ghost, destructive, link.

13.4 Полета

`input("име", "плейсхолдър", "kind")`. Видове: text, number, password, email, url.

13.5 Многоредови полета

`textarea("body", "...", 10)`.

13.6 Чекбоксове и радиа

`checkbox(bind, label)`, `radio(bind, value, label)`.

13.7 Слайдъри и select

`slider`, `select`.

13.8 Таблици

`table(["Име","Възраст"], [["Ада",36]])`.

13.9 Tabs

tabs/endtabs + tab/endtab.

13.10 Менюта

menu/endmenu + menuitem.

13.11 Подредба

row/endrow, column/endcolumn, group/endgroup.

13.12 Реактивни стойности

`uiget(name)` и `uiset(name, value)`.

13.13 Събития

Бутоните и менютата приемат callback.

13.14 Персонализация

Дикционар с опции като последен аргумент: размер, икона, URL.

13.15 Пълен пример

```
forge todos be []
window("Todos", 360, 480)
  input("new", "Какво да направя?")
```

```
    button("Добави", () =&gt; { gather(todos, uiget("new")); uiset("n
    for t in todos: label("• " + t) end
  endwindow
```

14. Графика и canvas

2D рисуване и анимация.

14.1 Canvas

`canvas(width, height)`.

14.2 Примитиви

`fill`, `stroke`, `rect`, `circle`, `line`, `polygon`, `text`.

14.3 Цветове

CSS имена, `#rrggbb`, `rgb()`, `hsl()`, `rgba()`.

14.4 Анимация

`frame(fn)`.

14.5 Изображения

`loadImage`, `drawImage`.

14.6 Трансформации

`translate`, `rotate`, `scale`, `push`, `pop`.

14.7 Вход

`onmouse`, `onkey`.

14.8 Подскачаща топка

```
canvas(400, 400)
forge x be 200; forge y be 200; forge dx be 3; forge dy be 2
frame((t) =&gt; {
  fill("black"); rect(0, 0, 400, 400)
  fill("cyan"); circle(x, y, 20)
  x be x + dx; y be y + dy
})
```

15. Карти и ГИС

Интеграция с Leaflet.

15.1 Карта

`map(lat, lon, zoom)`.

15.2 Маркери

`marker(lat, lon, "етикет")`.

15.3 Tile слоеве

`tiles(url, opts)`. Стилове: osm, dark, satellite.

15.4 Полилинии

`polyline([[lat,lon],...], "color")`.

15.5 Полигони

`polygon([[lat,lon],...], "color")`.

15.6 GeoJSON

`geojson(url)`.

15.7 Heatmaps

`heatmap(points)`, `cluster(points)`.

15.8 Събития

`onclick((lat, lon) => ...)`.

15.9 Маршрутиране

`distance(p1, p2)`, `route(from, to)`.

16. Стандартна библиотека

Справочник.

16.1 Таблица

| Функция | Описание | |---|---| | `speak(value)` | Print a value followed by a newline. | | `whisper(value)` | Print a value with no newline. | | `shout(value)` | Print a value in uppercase, attention-grabbing form. | | `ask(prompt)` | Read a line from the user (CLI) or prompt dialog (web). | | `readFile(path)` | Read the entire contents of a text file as a string. | | `writeFile(path, text)` | Write a string to a file, replacing previous contents. | | `appendFile(path, text)` | Append a string to the end of a file. | | `exists(path)` | Returns yes if the file or folder exists. | | `delete(path)` | Removes a file or folder. | | `listdir(path)` | Returns a collection of names inside a folder. | | `essence(value)` | Returns the runtime type as a string. | | `morph(value, type)` | Convert a value to another type. | | `magnitude(value)` | Length of a string or collection; absolute value of a number. | | `least(...)` | Smallest of the given values or collection. | | `greatest(...)` | Largest of the given values or collection. | | `root(n, k=2)` | k-th root of n; default square root. | | `ground(n)` | Floor; round toward negative infinity. | | `elevate(n)` | Ceiling; round toward positive infinity. | | `around(n, digits=0)` | Round to a number of decimal digits. | | `random()` | Random float in [0, 1). | | `randint(a, b)` | Random integer in [a, b]. | | `choose(coll)` | Random element from a collection. |

| `sin(x) / cos(x) / tan(x)` | Trigonometry in radians. | | `asin/acos/atan/atan2` | Inverse trig. | | `log(x, base=e)` | Logarithm. | | `exp(x)` | Exponential e^x. | | `pi / tau / e` | Math constants. | | `gather(coll, value)` | Append a value to a collection in place. | | `pluck(coll, index)` | Remove and return an element at index. | | `portion(coll, start, end?)` | Slice a collection. | | `each(coll, fn)` | Apply fn to each element. | | `sift(coll, predicate)` | Filter a collection. | | `fold(coll, init, fn)` | Reduce a collection to a single value. | | `seek(coll, predicate)` | First element matching predicate. | | `every(coll, predicate)` | Yes if all elements match. | | `some(coll, predicate)` | Yes if any element matches. | | `zip(a, b, ...)` | Pairs elements from multiple collections. | | `enumerate(coll)` | Pairs of (index, element). | | `sequence(start, end, step=1)` | A range of numbers. | | `sort(coll, key?)` | Sort a collection ascending. |

16.2 I/O

`speak`, `whisper`, `shout`, `ask`, `readFile`, `writeFile`, `appendFile`.

16.3 Математика

`magnitude`, `least`, `greatest`, `root`, `random`, `sin`, `cos`, `pi`.

16.4 Колекции

`gather`, `pluck`, `each`, `sift`, `fold`, `zip`, `enumerate`.

16.5 Низове

`upper`, `lower`, `trim`, `replace`, `shatter`, `weave`.

16.6 Време

`now`, `today`, `delay`, `format`.

16.7 Мрежа

`fetch`, `websocket`, `download`.

16.8 Система

`env`, `argv`, `exit`, `osinfo`.

16.9 Кодиране

`base64encode`, `hexencode`, `jsonencode`.

16.10 Криптография

`hash`, `hmac`, `uuid`, `randomBytes`.

17. Инструменти

Редактори, линтер, дебъгер, компилатор.

17.1 Уеб IDE

Пълнофункционално IDE на https://s-dev.lovable.app/ide.

17.2 VS Code разширение

Изтеглете `.vsix`.

17.3 CLI

Флагове: `--repl`, `--compile`, `--exec`, `--lang`, `--ast`, `--time`.

17.4 Bytecode компилатор

`--compile main.sdev` → `main.sdevc`.

17.5 sdev асистент

Вграден AI чатбот.

17.6 Линтер

`sdev lint file.sdev`.

17.7 Форматер

`sdev format file.sdev`.

17.8 Дебъгер

`breakpoint()` пауза в IDE.

17.9 Тестове

`sdev test` пуска `conjure test_*` функции.

18. Напреднали теми

Функции за опитни потребители.

18.1 JavaScript Interop

`js { console.log("native") }`. Многоредови блокове споделят обхват.

18.2 Матрична математика

`matrix`, `matmul`, `inverse`, `transpose`, `det`.

18.3 Буфери и указатели

`buffer(размер)`, `pointer(стойност)`, `deref(p)`.

18.4 Виртуално ядро

`spawn(fn)`, `schedule()`, `syscall("име", args)`.

18.5 Сметоизхвърляне

Mark-and-sweep GC. Принудете цикъл с `gc()`. `heapinfo()`.

18.6 Bytecode VM

Стек-базирана VM. Опкодове: LOAD_CONST, CALL, JUMP, RETURN, MAKE_FUNCTION.

18.7 Многоезичен код

26 езика. `// lang: bg` директива.

18.8 Вграждане

`import { run } from "sdev-interpreter.js"` в JS, `from sdev_interpreter import run` в Python.

18.9 FFI

`ffi.load("libm.so").func(...)`.

19. Производителност

Как да направите sdev по-бърз, когато е необходимо.

19.1 Изберете рънтайм

Tree-walker: най-бавен. Bytecode: 3-5× по-бърз. JS interop: native.

19.2 Профилирайте

`time { ... }`, `profile { ... }`.

19.3 Избягвайте квадратични шаблони

`pluck(coll, 0)` в цикъл е $O(n^2)$. Използвайте `deque`.

19.4 Кеширане

`@memo` за чисти функции.

19.5 Преизползвайте буфери

Един `buffer(N)` извън цикъла.

19.6 Компилирайте

За дълготрайни процеси, `--compile`.

20. Добри практики

Идиоми и шаблони.

20.1 Именувайте добре

sdev възнаграждава описателните имена.

20.2 Предпочитайте глаголи

Имена на функции = глаголи; имена на класове = съществителни.

20.3 Малки функции

Под 20 реда.

20.4 Прегърнете неизменността

Използвайте `eternal` често.

20.5 Документирайте намерението

Коментарите обясняват *защо*.

20.6 Тествайте рано

Един `expect` на функция.

20.7 Проваляйте се шумно

Валидирайте на границите.

20.8 Композиция над наследяване

Mixins (`weave`) надживяват дълбоки йерархии.

21. Готварска книга с примери

Кратки, пълни програми.

21.1 FizzBuzz

```
for i in sequence(1, 100):
  when:
    is i % 15 == 0 then speak("FizzBuzz")
    is i % 3 == 0 then speak("Fizz")
    is i % 5 == 0 then speak("Buzz")
    else speak(i)
  end
end
```

21.2 Брояч на думи

```
forge text be readFile("essay.txt")
speak(magnitude(shatter(text, " ")))
```

21.3 Хистограма

```
forge counts be {}
for w in shatter(readFile("text.txt"), " "):
  counts[w] be (counts[w] or 0) + 1
end
```

21.4 Todo приложение

```
forge todos be []
window("Todos", 360, 480)
  input("new", "Какво да направя?")
  button("Добави", () =&gt; { gather(todos, uiget("new")); uiset("n
  for t in todos: label(t) end
endwindow
```

21.5 HTTP заявка

```
async conjure main():
  forge res be await fetch("https://api.example.com")
  speak(res.body)
end
await main()
```

21.6 Подскачаща топка

```
canvas(400, 400)
forge x be 200; forge y be 200; forge dx be 3; forge dy be 2
frame((t) =&gt; { fill("black"); rect(0,0,400,400); fill("cyan"); c
```

21.7 Палиндром

```
conjure palindrome(s):
  forge clean be lower(replace(s, " ", ""))
  return clean equals reverse(clean)
```

```
end
```

21.8 Фибоначи

```
@memo
conjure fib(n):
  if n below 2 then return n end
  return fib(n-1) + fib(n-2)
end
```

21.9 Калкулатор UI

```
window("Calc", 280, 320)
  input("a", "Първо", "number")
  input("b", "Второ", "number")
  button("+", () =&gt; uiset("r", uiget("a") + uiget("b")))
  label("= " + (uiget("r") or "?"))
endwindow
```

21.10 Карта на столици

```
map(48.0, 10.0, 4)
for (c, lat, lon) in [("София",42.7,23.3),("Берлин",52.5,13.4)]:
  marker(lat, lon, c)
end
```

22. Изграждане на реално приложение

От празен файл до работещ Markdown notebook.

22.1 Подредба

`main.sdev`, `lib/storage.sdev`, `lib/render.sdev`, `ui/notebook.sdev`.

22.2 Storage модул

```
share conjure save(key, text):
  writeFile("notes/" + key + ".md", text)
end
```

22.3 Render модул

Малък регекс конвертор или `summon "gist:abc/markdown.sdev"`.

22.4 UI

```
window("Notebook", 720, 600)
  input("title", "Заглавие")
  textarea("body", "Пишете...", 12)
  button("Запис", () =&gt; storage.save(uiget("title"), uiget("body
endwindow
```

22.5 Свързване

`main.sdev` импортира модулите.

22.6 Облак

Когато сте логнати, работното пространство автоматично синхронизира.

22.7 Публикуване

`sdev publish` качва като gist.

23. Често задавани въпроси

Реални въпроси от реални потребители.

23.1 Бърз ли е sdev?

Сравнима с Python производителност. Bytecode дава 3-5× ускорение.

23.2 Може ли в продукция?

Подходящ за скриптове, образование, прототипи, вътрешни инструменти.

23.3 Защо не Python?

sdev съществува за тези, които искат *по-четим* алтернативен синтаксис.

23.4 Защо два рънтайма?

JavaScript за уеб, Python за научни среди.

23.5 Бъгове?

GitHub issue.

23.6 Принос?

Да, sdev е с отворен код.

23.7 Запазване между сесии?

Влезте в акаунт. IDE автоматично синхронизира.

23.8 Защо "forge"?

Активен глагол; "let" е пасивен.

24. Речник

Бързи дефиниции.

24.1 А–Й

Анонимна функция — без име. **Bytecode** — компактният формат от `--compile`. **Затваряне** — функция, която улавя външни променливи. **Eternal** — модификатор за неизменими обвързвания.

24.2 К–Р

Колекция — списък или речник. **Lambda** — синоним за анонимна функция. **Memo** — декоратор за кеширане. **Модул** — `.sdev` файл. **Морф** — за конверсия на типове.

24.3 С–Я

Обект — инстанция на клас. **Обещание** — стойност, която ще пристигне по-късно. **Реактивна стойност** — UI-обвързана стойност. **Tail call** — извикване в опашка; оптимизирано. **BM** — байткод виртуалната машина.

25. Бележки за дизайна

Защо sdev изглежда така.

25.1 Глаголи над символи

Символите четат бързо за експерти и бавно за нови. Глаголите четат приятно.

25.2 Защо `end`

Значимите отстъпи са крехки в редактори и чатове. `end` оцелява copy-paste.

25.3 Два рънтайма

JavaScript и Python покриват повечето среди.

25.4 UI вграден

Език без бутон е език за уроци.

25.5 Многоезичен

Програмирането не трябва да е блокирано от английски.

25.6 Вдъхновения

Lua, Python, Smalltalk, Ruby, AppleScript.

26. Приложения

26.1 A. Ключови думи

`forge`, `be`, `eternal`, `if`, `then`, `else`, `when`, `is`, `while`, `until`, `for`, `in`, `escape`, `skip`, `conjure`, `yield`, `return`, `essence`, `summon`, `share`, `extends`, `me`, `ancestor`, `attempt`, `rescue`, `finally`,

`raise`, `dispatch`, `await`, `js`, `end`, `window`, `endwindow`, `row`, `endrow`, `column`, `endcolumn`, `tabs`, `endtabs`, `tab`, `endtab`, `group`, `endgroup`, `menu`, `endmenu`, `menuitem`

26.2 B. Оператори

`+ - * / % ** //`, `== != < > <= >=`, `and or not`, `equals differs above below`, `in`, `=>`, `|>`.

26.3 C. Вградени функции

Виж глава "Стандартна библиотека".

26.4 D. Изходни езици

26 езика, включително български, английски, испански, френски, немски, руски, китайски, японски, корейски и арабски.

26.5 E. Bytecode опкодове

LOAD_CONST, LOAD_NAME, STORE_NAME, CALL, JUMP, JUMP_IF_FALSE, RETURN, MAKE_FUNCTION, BUILD_LIST, BUILD_DICT, IMPORT, RAISE, YIELD, AWAIT, SPAWN, HALT.

26.6 F. Syscalls

open, close, read, write, sleep, fork, socket, connect, time, gc.

26.7 G. Кодове на грешки

E_TYPE, E_VALUE, E_INDEX, E_KEY, E_IO, E_NET, E_TIMEOUT, E_SYNTAX.

26.8 H. CLI флагове

--repl, --compile, --exec, --lang, --ast, --time, --profile, --lint, --format, --test.

26.9 I. Индекс на примерите

Подскачаща топка, Калкулатор, Карта на столици, FizzBuzz, Хистограма, Notebook, Палиндром, Todo, Web scraper, Брояч на думи.

Край на книгата. Версия 2.0. © sdev език.